Midterm

Tiffany Le

2025-03-26

```
#remove.packages("h2o")
#install.packages("h2o")
```

Prerequisites

```
# Helper packages
library(rsample)   # for creating our train-test splits
```

```
## Warning: package 'rsample' was built under R version 4.3.2
```

```
library(recipes)   # for minor feature engineering tasks
```

```
## Warning: package 'recipes' was built under R version 4.3.3
```

```
## Loading required package: dplyr
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
##
## Attaching package: 'recipes'
```

```
## The following object is masked from 'package:stats':
##
##     step
```

```
library(dplyr)    # for data manipulation
library(ggplot2)  # for data visualization
library(tidyr)    # for data reshaping

# Modeling packages
library(h2o)       # for fitting stacked models
```

```
##
## ----------------------------------------------------------------------
##
## Your next step is to start H2O:
##     > h2o.init()
```

```
##
## For H2O package documentation, ask for help:
##     > ??h2o
##
## After starting H2O, you can use the Web UI at http://localhost:54321
## For more information visit https://docs.h2o.ai
##
## ----------------------------------------------------------------------
```

```
##
## Attaching package: 'h2o'
```

```
## The following objects are masked from 'package:stats':
##
##     cor, sd, var
```

```
## The following objects are masked from 'package:base':
##
##     &&, %*%, %in%, ||, apply, as.factor, as.numeric, colnames,
##     colnames<-, ifelse, is.character, is.factor, is.numeric, log,
##     log10, log1p, log2, round, signif, trunc
```

```
library(recipes)   # for ML recipes
library(xgboost)   # for fitting GBMs
```

```
## Warning: package 'xgboost' was built under R version 4.3.3
```

```
##
## Attaching package: 'xgboost'
```

```
## The following object is masked from 'package:dplyr':
##
##     slice
```

```
h2o.no_progress()  # turn off progress bars
h2o.init(max_mem_size = "5g")  # connect to H2O instance
```

```
##  Connection successful!
##
## R is connected to the H2O cluster:
##     H2O cluster uptime:         5 hours 19 minutes
##     H2O cluster timezone:       America/Los_Angeles
##     H2O data parsing timezone:  UTC
##     H2O cluster version:        3.44.0.3
##     H2O cluster version age:    1 year, 4 months and 11 days
##     H2O cluster name:           H2O_started_from_R_tifle_nit536
##     H2O cluster total nodes:    1
##     H2O cluster total memory:   2.11 GB
##     H2O cluster total cores:    8
##     H2O cluster allowed cores:  8
##     H2O cluster healthy:        TRUE
##     H2O Connection ip:          localhost
##     H2O Connection port:        54321
##     H2O Connection proxy:       NA
##     H2O Internal Security:      FALSE
##     R Version:                  R version 4.3.1 (2023-06-16)
```

```
## Warning in h2o.clusterInfo():
```

```
## Your H2O cluster version is (1 year, 4 months and 11 days) old. There may be a newer version availabl
## Please download and install the latest version from: https://h2o-release.s3.amazonaws.com/h2o/latest_
```

Ames Housing Dataset

```
# Load and split the Ames housing data
ames <- AmesHousing::make_ames()
set.seed(123)  # for reproducibility
split <- initial_split(ames, strata = "Sale_Price")
ames_train <- training(split)
ames_test <- testing(split)

# Make sure we have consistent categorical levels
blueprint <- recipe(Sale_Price ~ ., data = ames_train) %>%
  step_other(all_nominal(), threshold = 0.005)

# Create training & test sets for h2o
train_h2o <- prep(blueprint, training = ames_train, retain = TRUE) %>%
  juice() %>%
  as.h2o()
test_h2o <- prep(blueprint, training = ames_train) %>%
  bake(new_data = ames_test) %>%
  as.h2o()

# Get response and feature names
Y <- "Sale_Price"
X <- setdiff(names(ames_train), Y)
```

Base Learner - Models

Regularized Regression Base Learner

Random Forest Base Learner

```
# Train & cross-validate a RF model
best_rf <- h2o.randomForest(
  x = X, y = Y, training_frame = train_h2o, ntrees = 100, mtries = -1,
  max_depth = 15, min_rows = 1, sample_rate = 0.8, nfolds = 2,
  fold_assignment = "Modulo", keep_cross_validation_predictions = TRUE,
  seed = 123, stopping_rounds = 50, stopping_metric = "RMSE",
  stopping_tolerance = 0
)
```

```
## Warning in .h2o.processResponseWarnings(res): early stopping is enabled but neither score_tree_interv
```

Gradient Boosting Base Learner

```
# Train & cross-validate a GBM model
best_gbm <- h2o.gbm(
  x = X, y = Y, training_frame = train_h2o, ntrees = 500, learn_rate = 0.01,
  max_depth = 7, min_rows = 5, sample_rate = 0.8, nfolds = 2,
  fold_assignment = "Modulo", keep_cross_validation_predictions = TRUE,
  seed = 123, stopping_rounds = 50, stopping_metric = "RMSE",
  stopping_tolerance = 0
```

```
)
```

```
## Warning in .h2o.processResponseWarnings(res): early stopping is enabled but neither score_tree_interv
```

XGBoost Base Learner

```
# Train & cross-validate an XGBoost model
best_xgb <- h2o.xgboost(
  x = X, y = Y, training_frame = train_h2o, ntrees = 5000, learn_rate = 0.05,
  max_depth = 3, min_rows = 3, sample_rate = 0.8, categorical_encoding = "LabelEncoder",
  nfolds = 2, fold_assignment = "Modulo",
  keep_cross_validation_predictions = TRUE, seed = 123, stopping_rounds = 50,
  stopping_metric = "RMSE", stopping_tolerance = 0
)
```

```
## Warning in .h2o.processResponseWarnings(res): early stopping is enabled but neither score_tree_interv
```

Neural Network Stack Model (Super Learner)

Results

```
# Get results from base learners
get_rmse <- function(model) {
  results <- h2o.performance(model, newdata = test_h2o)
  results@metrics$RMSE
}
list(best_glm, best_rf, best_gbm, best_xgb) %>%
  purrr::map_dbl(get_rmse)
```

```
## [1] 30018.69 22970.61 21127.53 19968.97
```

```
# Stacked results
h2o.performance(ensemble_nn, newdata = test_h2o)@metrics$RMSE
```

```
## [1] 20600.09
```

```
data.frame(
  GLM_pred = as.vector(h2o.getFrame(best_glm@model$cross_validation_holdout_predictions_frame_id$name)),
  RF_pred = as.vector(h2o.getFrame(best_rf@model$cross_validation_holdout_predictions_frame_id$name)),
  GBM_pred = as.vector(h2o.getFrame(best_gbm@model$cross_validation_holdout_predictions_frame_id$name)),
  XGB_pred = as.vector(h2o.getFrame(best_xgb@model$cross_validation_holdout_predictions_frame_id$name))
) %>% cor()
```

```
##           GLM_pred   RF_pred  GBM_pred  XGB_pred
## GLM_pred 1.0000000 0.9179932 0.9147457 0.9151065
## RF_pred  0.9179932 1.0000000 0.9918391 0.9793478
## GBM_pred 0.9147457 0.9918391 1.0000000 0.9783649
## XGB_pred 0.9151065 0.9793478 0.9783649 1.0000000
```

Visualization / Comparison of Models

```
# Create a data frame with all model results
model_comparison <- data.frame(
  Model = c("GLM", "Random Forest", "GBM", "XGBoost", "Neural Network Ensemble"),
  RMSE = c(
```

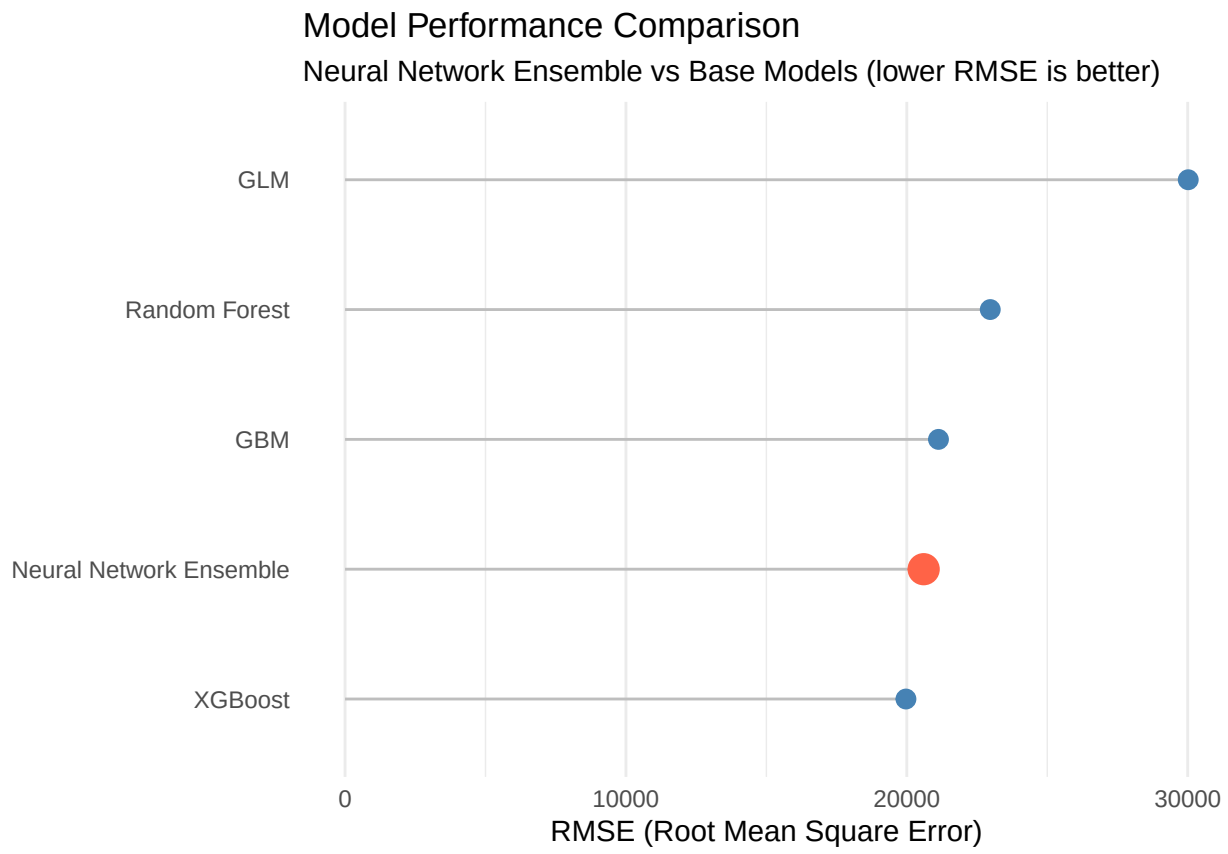
```
    get_rmse(best_glm),
    get_rmse(best_rf),
    get_rmse(best_gbm),
    get_rmse(best_xgb),
    h2o.performance(ensemble_nn, newdata = test_h2o)@metrics$RMSE
  )
)

# Sort models by performance (lowest RMSE first)
model_comparison <- model_comparison[order(model_comparison$RMSE), ]

# Show Model Comparison
model_comparison
```

```
##                       Model     RMSE
## 4                   XGBoost 19968.97
## 5 Neural Network Ensemble 20600.09
## 3                       GBM 21127.53
## 2           Random Forest 22970.61
## 1                       GLM 30018.69
```

```
# Compare Ensemble to Others
ggplot(model_comparison, aes(x = reorder(Model, RMSE))) +
  geom_segment(aes(xend = Model, y = 0, yend = RMSE), color = "gray") +
  geom_point(aes(y = RMSE, color = Model == "Neural Network Ensemble",
                 size = Model == "Neural Network Ensemble")) +
  scale_color_manual(values = c("steelblue", "tomato"), guide = "none") +
  scale_size_manual(values = c(3, 5), guide = "none") +
  coord_flip() +
  theme_minimal() +
  labs(title = "Model Performance Comparison",
       subtitle = "Neural Network Ensemble vs Base Models (lower RMSE is better)",
       y = "RMSE (Root Mean Square Error)",
       x = NULL) +
  theme(panel.grid.major.y = element_blank())
```

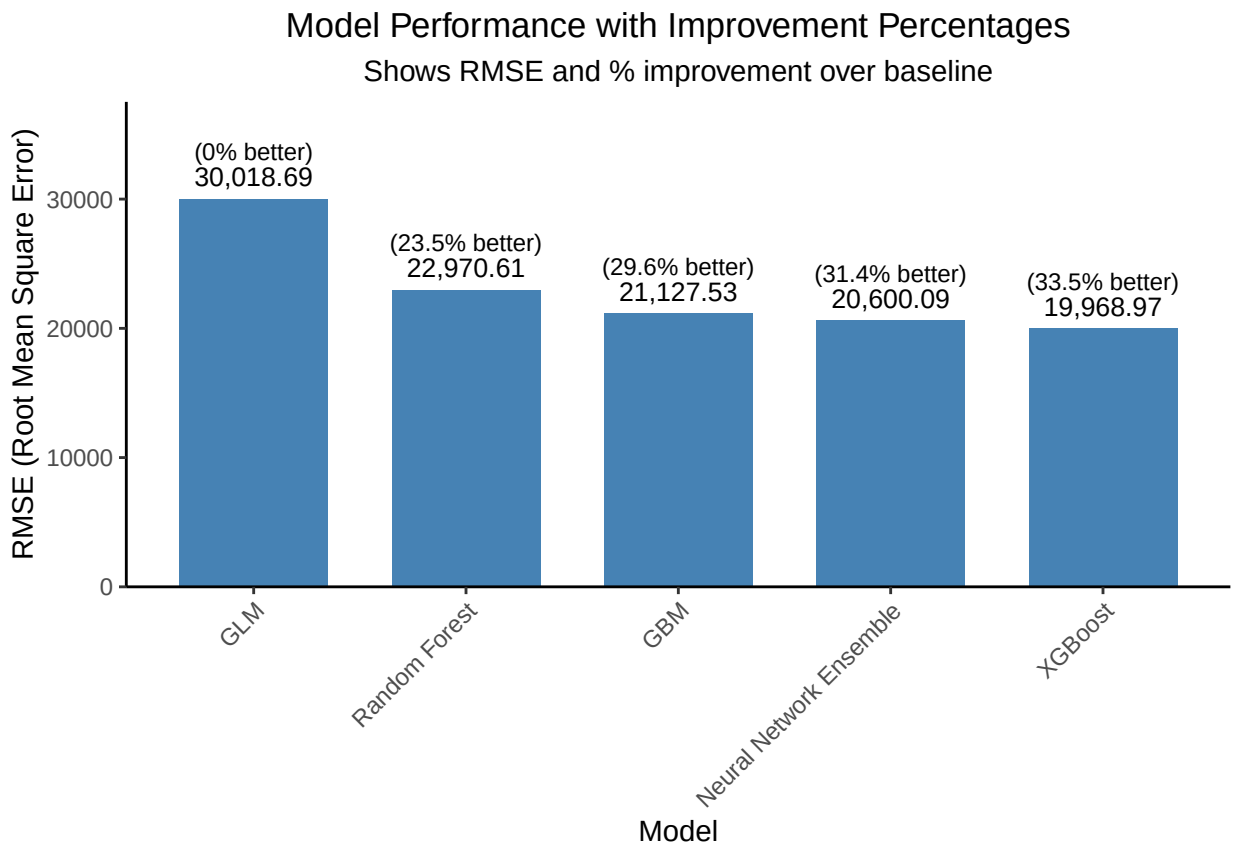



```r
# Calculate percentage improvement over worst model
baseline_rmse <- max(model_comparison$RMSE)
model_comparison$Improvement <- (baseline_rmse - model_comparison$RMSE) / baseline_rmse * 100

# Add percentage improvement labels to the comparison
clean_chart <- ggplot(model_comparison, aes(x = reorder(Model, -RMSE), y = RMSE)) +
  geom_bar(stat = "identity", width = 0.7,
           fill = ifelse(model_comparison$Model == "NN_Ensemble", "tomato", "steelblue")) +
  # Position labels ABOVE the bars with proper spacing
  geom_text(aes(label = format(round(RMSE, 2), big.mark=","),
                y = RMSE + 1000),       # Position above bars with 1000 units of space
            vjust = 0,
            size = 3.5) +
  # Add smaller improvement text below first text
  geom_text(aes(label = paste0("(", round(Improvement, 1), "% better)"),
                y = RMSE + 1000),
            vjust = -1.5,
            size = 3) +
  theme_classic() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1),
        plot.title = element_text(hjust = 0.5),
        plot.subtitle = element_text(hjust = 0.5)) +
  labs(title = "Model Performance with Improvement Percentages",
       subtitle = "Shows RMSE and % improvement over baseline",
       x = "Model",
       y = "RMSE (Root Mean Square Error)") +
```

```
  scale_y_continuous(limits = c(0, max(model_comparison$RMSE) * 1.25),  # Add 25% space at top
                     expand = c(0, 0))  # Remove default expansion

print(clean_chart)
```




```
# Function to get RMSE from H2O models
get_rmse <- function(model) {
  results <- h2o.performance(model, newdata = test_h2o)
  results@metrics$RMSE
}

# Get RMSE for all models
rmse_values <- c(
  GLM = get_rmse(best_glm),
  RF = get_rmse(best_rf),
  GBM = get_rmse(best_gbm),
  XGBoost = get_rmse(best_xgb),
  NN_Ensemble = h2o.performance(ensemble_nn, newdata = test_h2o)@metrics$RMSE
)

# Create a data frame with all model results
model_comparison <- data.frame(
  Model = names(rmse_values),
  RMSE = unname(rmse_values)
)
```

Comparison Table

```
# Sort models by performance (lowest RMSE first)
model_comparison <- model_comparison[order(model_comparison$RMSE), ]

# Calculate percentage improvement over worst model (highest RMSE)
baseline_rmse <- max(model_comparison$RMSE)
model_comparison$Improvement <- round((baseline_rmse - model_comparison$RMSE) / baseline_rmse * 100, 2)

# Calculate percentage improvement over best base model
best_base_rmse <- min(model_comparison$RMSE[model_comparison$Model != "NN_Ensemble"])
model_comparison$Improvement_Over_Best_Base <- NA
model_comparison$Improvement_Over_Best_Base[model_comparison$Model == "NN_Ensemble"] <-
  round((best_base_rmse - model_comparison$RMSE[model_comparison$Model == "NN_Ensemble"]) / best_base_r

# Create a nicely formatted performance comparison table
print("Model Performance Comparison (lower RMSE is better)")
```

```
## [1] "Model Performance Comparison (lower RMSE is better)"
```

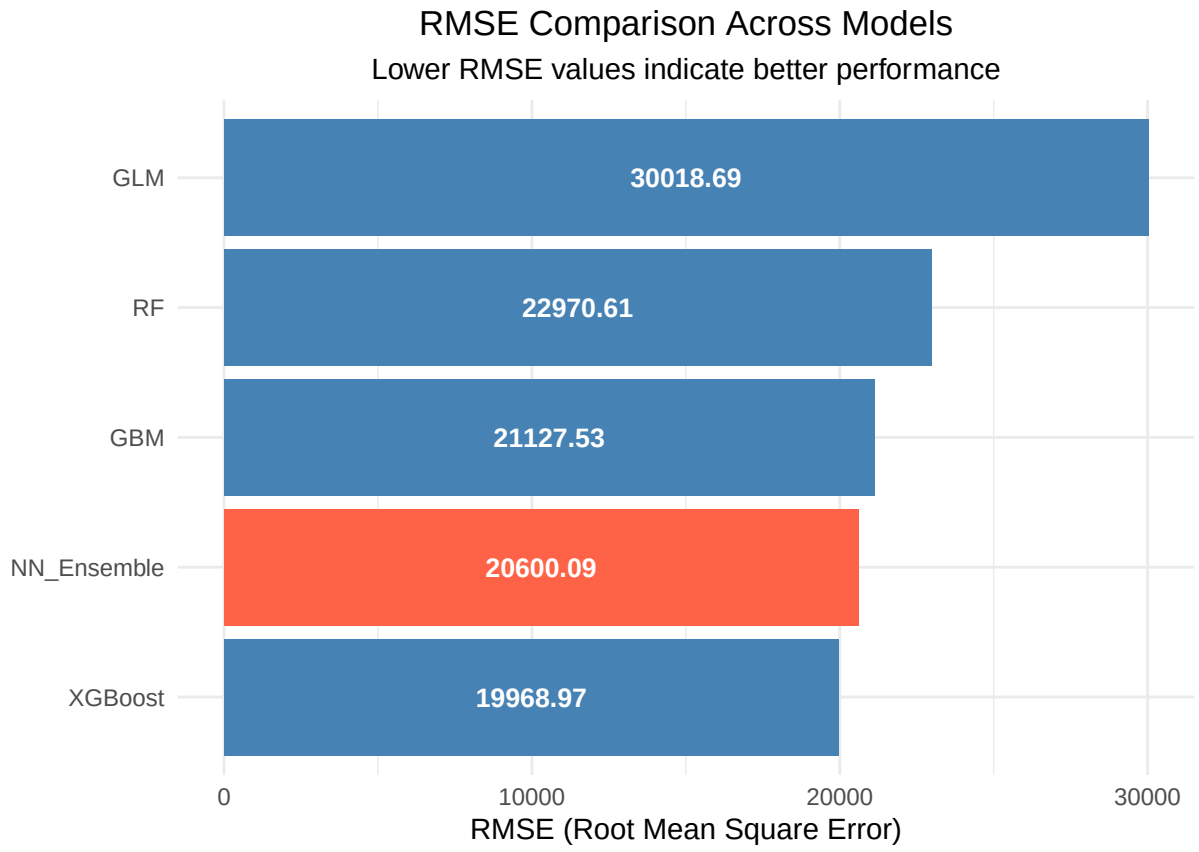
```
print(model_comparison)
```

```
##         Model     RMSE Improvement Improvement_Over_Best_Base
## 4     XGBoost 19968.97       33.48                        NA
## 5 NN_Ensemble 20600.09       31.38                     -3.16
## 3         GBM 21127.53       29.62                        NA
## 2          RF 22970.61       23.48                        NA
## 1         GLM 30018.69        0.00                        NA
```

Comparison via Bar Chart (Red for Stacked NN)

```
# Basic bar chart of RMSE values
bar_chart <- ggplot(model_comparison, aes(x = reorder(Model, RMSE), y = RMSE)) +
  geom_bar(stat = "identity",
           aes(fill = Model == "NN_Ensemble"),
           show.legend = FALSE) +
  scale_fill_manual(values = c("steelblue", "tomato")) +
  geom_text(aes(label = round(RMSE, 2)),
            position = position_stack(vjust = 0.5),  # Position text in middle of bars
            color = "white",                         # White text for better contrast
            fontface = "bold", size = 3.5) +         # Bold text
  coord_flip() +                                     # Flip for horizontal bars
  theme_minimal() +
  labs(title = "RMSE Comparison Across Models",
       subtitle = "Lower RMSE values indicate better performance",
       x = "",
       y = "RMSE (Root Mean Square Error)") +
  theme(plot.title = element_text(hjust = 0.5),
        plot.subtitle = element_text(hjust = 0.5))

bar_chart
```



Summary Statistics

```
# Identify best model
best_model <- model_comparison$Model[1]
best_rmse <- model_comparison$RMSE[1]

# Find the best base model (not ensemble)
best_base_model <- model_comparison$Model[model_comparison$Model != "NN_Ensemble"][1]
best_base_rmse <- model_comparison$RMSE[model_comparison$Model == best_base_model]

# Calculate improvement of ensemble over best base model
if (best_model == "NN_Ensemble") {
  ensemble_improvement <- ((best_base_rmse - best_rmse) / best_base_rmse) * 100
  ensemble_status <- paste("The Neural Network Ensemble improved upon the best base model by",
                           round(ensemble_improvement, 2), "%")
} else {
  ensemble_status <- "The Neural Network Ensemble did not outperform the best base model"
}

# Print summary
cat("\n===== SUMMARY =====\n")
```

```
##
## ===== SUMMARY =====
```

```
cat("Best overall model:", best_model, "with RMSE =", round(best_rmse, 3), "\n")
```

```
## Best overall model: XGBoost with RMSE = 19968.97
```

```r
cat("Best base model:", best_base_model, "with RMSE =", round(best_base_rmse, 3), "\n")
```

```
## Best base model: XGBoost with RMSE = 19968.97
```

```r
cat(ensemble_status, "\n")
```

```
## The Neural Network Ensemble did not outperform the best base model
```

```r
cat("======================\n")
```

```
## ======================
```